fsd-creator

SKILL.md — Full Reference Document

Trigger keywords: **createfsd** · **newproject** · **newrequirements**

Output format: Markdown (.md) · Audience: developers / engineers · Requirements: tagged REQ-### / TEST-###

Skill Description (YAML frontmatter)

```
name: fsd-creator
description: >
Generates a Feature Specification Document (FSD) in Markdown format from a
high-level
project description. Use this skill whenever the user says "createfsd",
"newproject",
or "newrequirements", or asks to generate a feature spec, requirements
document, project
spec, or FSD. Also trigger when the user describes a new software project and
wants
documentation, requirements, or a spec produced - even if they don't use those
exact keywords.
Always use this skill when any of these triggers appear, even mid-conversation.
```

Overview

Generates a structured Feature Specification Document (FSD) in Markdown format for a new software project. Targets a developer/engineer audience. Always numbers and tags requirements for traceability (REQ-001, REQ-002, etc.).

Workflow

STEP 1 Capture the Initial Description

The user will have provided a high-level project description (or a trigger keyword with no description yet). Either way, proceed to the interview before generating anything.

STEP 2 Conduct the Interview

Ask the user the following questions before generating the FSD. Group them into a single message so the user can answer in one reply. Tailor which questions you ask based on what the initial description already answers — skip questions that are already clear.

Core questions to ask (as applicable):

1. **Project name** — What is this project called?
2. **Platform / target hardware** — What hardware or OS is this running on? (e.g., STM32F4, Raspberry Pi, Windows x64, bare-metal, RTOS, Linux)
3. **Language / toolchain** — Primary programming language(s) and compiler/IDE?

4. **Top-level goals** — What are the 2-4 most important things this system must do?
5. **External interfaces** — What does this system talk to? (sensors, APIs, buses like SPI/I2C/UART, cloud services, other software modules, etc.)
6. **Constraints** — Any hard constraints? (memory limits, timing/latency, power budget, safety requirements, coding standards like MISRA, etc.)
7. **Out of scope** — Anything explicitly NOT part of this project?
8. **Revision info** — Author name and initial version number (default: v0.1)?

Only ask what isn't already answered by the description. If the project description is thorough, you may only need to ask 2-3 follow-up questions.

STEP 3 Generate the FSD

After the user replies to the interview, produce the FSD as a Markdown file saved to the output directory. Use the structure below exactly, in this order.

FSD Document Structure

```
# [Project Name] -- Feature Specification Document

## Revision History
## 1. Project Overview
## 2. Functional Requirements
## 3. Hardware & Platform Constraints
## 4. API & Interface Definitions
## 5. Test Plans
## Appendix (optional)
```

Section Details

Revision History

Version	Date	Author	Description
v0.1	YYYY-MM-DD	Name	Initial draft

Use today's date. Use the author and version from the interview (default v0.1).

1. Project Overview

- **Purpose**: 2-4 sentences describing what the project does and why it exists.
- **Scope**: What is included. A short 'Out of Scope' bullet list if applicable.
- **Assumptions & Dependencies**: External factors this project relies on.
- **Glossary**: Define domain-specific terms or acronyms (e.g., FW = firmware, HAL = Hardware Abstraction Layer).

2. Functional Requirements

Tag every requirement with a unique ID in the format REQ-### (zero-padded to 3 digits, starting at REQ-001). Group requirements into logical subsections.

Format each requirement as:

> REQ-001 - The system shall [do something specific and verifiable].
> Rationale: [brief reason why this requirement exists - optional but encouraged]

Rules for writing requirements:

- Use "shall" for mandatory requirements, "should" for desirable ones.
- Each requirement must be atomic — one testable condition per REQ.
- Avoid vague language ("fast", "easy", "reliable") — quantify where possible.
- Derive requirements from the user's goals and interview answers; infer reasonable requirements that a developer would expect even if not explicitly stated.

3. Hardware & Platform Constraints

Describe the target hardware and platform. Include:

- **Target hardware**: MCU/SoC/board name, architecture, clock speed if known.
- **Memory constraints**: Flash size, RAM size if known.
- **Operating environment**: RTOS, bare-metal, OS, toolchain, compiler.
- **Peripheral interfaces**: Which hardware peripherals are used (GPIO, SPI, I2C, UART, ADC, timers, etc.) and their role.
- **Power constraints**: If applicable.
- **Standards compliance**: MISRA, CERT-C, or other coding standards if applicable.

If a constraint directly drives a functional requirement, cross-reference it (e.g., 'See REQ-007').

4. API & Interface Definitions

Document the external and internal interfaces this project exposes or depends on.

4.1 External Hardware Interfaces — Describe physical bus protocols, pin assignments if known, data rates.

4.2 Software / Firmware APIs — List key functions, modules, or services this project will expose.

4.3 Communication Protocols — Describe any network, serial, or wireless protocols (e.g., MQTT, Modbus, BLE, CAN). Include message formats or packet structures if known.

4.4 Third-Party Libraries / SDKs — List any external dependencies (e.g., FreeRTOS, CMSIS, lwIP, a vendor HAL).

Example function signature format:

```
// Example
int sensor_read(sensor_t *dev, float *out_value);
```

If interface details are not yet known, write "TBD - to be defined in detailed design." Do not leave sections empty; note what is unknown.

5. Test Plans

Group tests to mirror the functional requirements. Tag each test with TEST-### (zero-padded, starting at TEST-001) and cross-reference the REQ it validates.

Format each test as:

> TEST-001 (validates REQ-001)
> Method: [Unit test / Integration test / Hardware-in-loop / Manual]
> Procedure: [Brief description of how to run the test]
> Pass Criteria: [Specific, measurable condition that constitutes a pass]

Include at minimum one test per top-level functional requirement group. Infer reasonable tests even if the user didn't specify them.

Appendix (optional)

Include if relevant:

- Reference documents or datasheets
- Related projects or predecessor systems
- Open questions / TBD items as a numbered list

STEP 4 Save and Present

Save the completed FSD as:

```
/mnt/user-data/outputs/[project-name]-fsd.md
```

Use lowercase, hyphenated project name (e.g., motor-controller-fsd.md).

Then use present_files to deliver it to the user.

After presenting, offer two follow-up options:

1. **Refine** — ask what sections or requirements they'd like to change.
2. **Export to Word** — mention they can ask for a .docx version if needed.

Quality Checklist

Self-review before saving the FSD. Verify all items pass:

- ■ Every REQ has a unique, sequential ID
- ■ Every TEST cross-references at least one REQ
- ■ No section is empty — "TBD" is acceptable, blank is not
- ■ All constraints from the interview appear somewhere in Section 3
- ■ All external interfaces from the interview appear in Section 4
- ■ Revision history has today's date filled in
- ■ Requirements use "shall" / "should" language, not vague adjectives

---

fsd-creator skill | Produced by Claude | Trigger: createfsd · newproject · newrequirements